Scrum Poker – Projektdokumentation

Modul	Web Technology (Leistungsnachweis «Scrum Poker App»)
Dozent	**TODO Gruppe: Name der Dozentin / des Dozenten eintragen**
Studiengang	**TODO Gruppe: Studiengang und Klasse eintragen**
Gruppenmitglieder	**TODO Gruppe: Vor- und Nachnamen aller Gruppenmitglieder eintragen**
Datum	25. Oktober 2026
Repository	https://github.com/gotazlib/abgaben/tree/main/web-technology/abgabe

Diese Dokumentation erklärt, wie die App funktioniert, wie sie aufgebaut ist und warum wir sie so gebaut haben. Die Kurzanleitung zum Starten steht im README. Verweise der Form `Datei:Zeile` zeigen auf den Code im Projektordner (in der Abgabe der Ordner `code/` bzw. der Inhalt von `scrum-poker-code.zip`). Zur Vorbereitung aufs Fachgespräch liegt dort auch der Leitfaden docs/FACHGESPRAECH.md (auch als PDF und HTML): ein Rundgang durch alle Dateien, Musterantworten, ein Demo-Drehbuch und Übungsaufgaben.

Wir haben die App mit den Mitteln aus dem Unterricht gebaut. Wo wir uns auf die Kursfolien beziehen, steht die Nummer des Foliensatzes und der Folie, z. B. «Folie 09/36» für Foliensatz 9 (React Hooks), Folie 36. Was nicht aus den Folien kommt, aber für die Aufgabe nötig war, ist ausdrücklich so markiert.

Inhalt

1. Funktionsweise
2. Architektur im Überblick
3. Komponentenstruktur und State
4. Ansichtswechsel mit State
5. REST-API
6. Datenfluss: Klick auf eine Karte
7. Lade- und Fehlerzustände
8. Strategie gegen Karteileichen
9. Durchschnitt bei T-Shirt-Grössen
10. Browser-Storage
11. Responsive Design und Box-Modell
12. Wie wir geprüft haben
13. Lokale Inbetriebnahme
14. Abgabe
15. KI-Einsatz

---

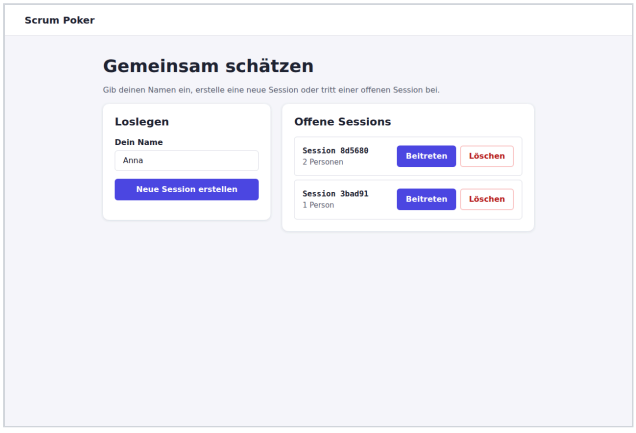
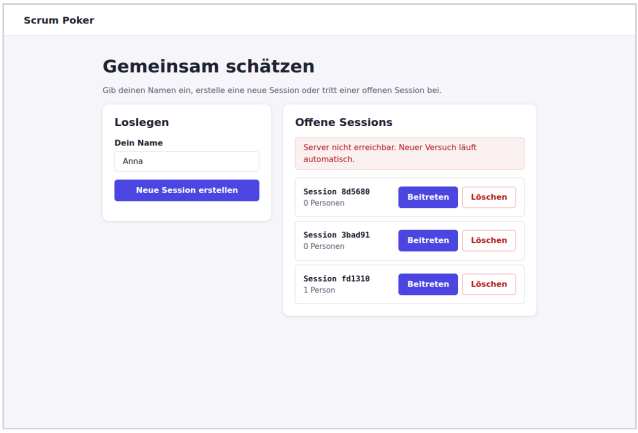
1. Funktionsweise

Scrum Poker (Planning Poker) ist eine Schätzmethode: Alle im Team legen gleichzeitig und verdeckt eine Karte mit ihrer Schätzung. Erst wenn alle gewählt haben, werden die Karten aufgedeckt. So lässt sich niemand von den anderen beeinflussen.

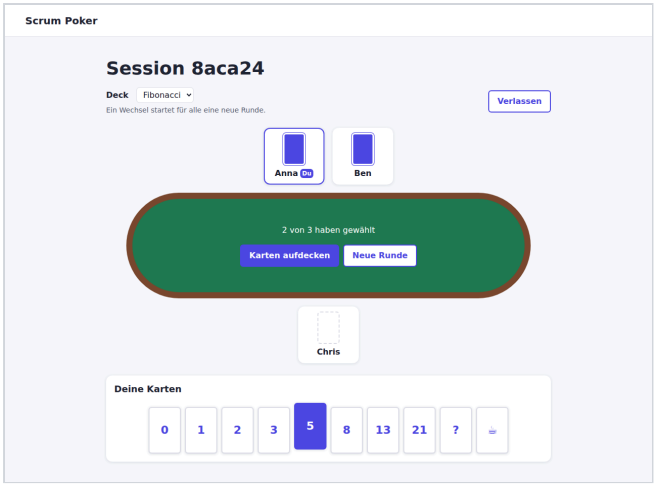
Ablauf in der App

1. **Name eingeben** auf der Startseite (`HomePage`). Der Name wird im Browser gespeichert und ist beim nächsten Besuch schon ausgefüllt. Ist der Name leer, besteht er nur aus Leerzeichen oder ist er länger als 30 Zeichen, erscheint eine Meldung, und «Neue Session erstellen» und «Beitreten» sind gesperrt.
2. **Session erstellen oder beitreten.** Die Liste «Offene Sessions» lädt sich alle 3 Sekunden neu und zeigt pro Session die ID und die Anzahl Personen. Sessions lassen sich nach einer Rückfrage löschen.
3. **Am Pokertisch** (`PokerTablePage`) sitzen alle Teilnehmenden um einen Tisch. Unten liegen die eigenen Karten. Ein Klick auf eine Karte gibt die Stimme ab, ein zweiter Klick auf dieselbe Karte zieht sie zurück. Die anderen sehen nur eine verdeckte Karte.
4. **Aufdecken:** von Hand mit «Karten aufdecken» oder automatisch, sobald alle gewählt haben. Danach zeigt der Tisch den Durchschnitt, die Anzahl Stimmen und gegebenenfalls «Konsens!».
5. **Neue Runde** löscht alle Stimmen und verdeckt den Tisch wieder.
6. **Deck wechseln** im Auswahlfeld «Deck» zwischen Fibonacci (0, 1, 2, 3, 5, 8, 13, 21, ?, ☕) und T-Shirt-Grössen (XS, S, M, L, XL, ?). Ein Wechsel startet für alle eine neue Runde.
7. **Verlassen** führt zurück zur Startseite und entfernt die Person sofort aus der Session. Wer den Tab schliesst, wird nach spätestens 20 s entfernt (Kapitel 8).
8. **Reload:** Nach einem Neuladen ist man wieder auf der Startseite, der Name ist noch da. So verlangt es der Leistungsnachweis (Folie 07/6: kein teilbarer Session-Link nötig, nach einem Reload bleibt nur der Name). Der alte Platz am Tisch verschwindet nach dem Timeout.

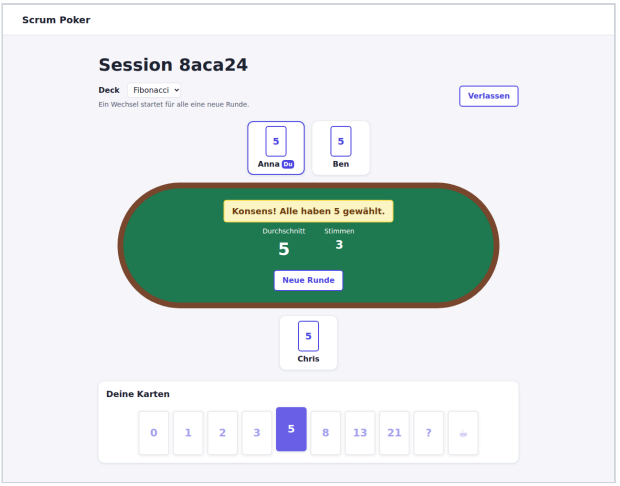
Am Tisch sehen alle Browserfenster Änderungen der anderen spätestens nach 1.5 Sekunden, in der Liste der Startseite nach 3 Sekunden (Polling).

Startseite (Desktop)	Server nicht erreichbar
	

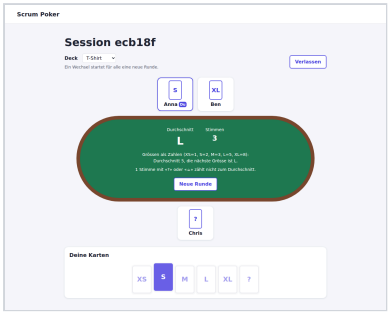
Pokertisch, verdeckt



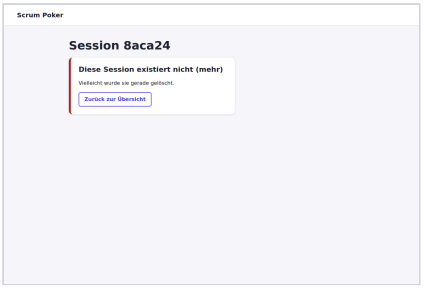
Nach dem Aufdecken mit Konsens



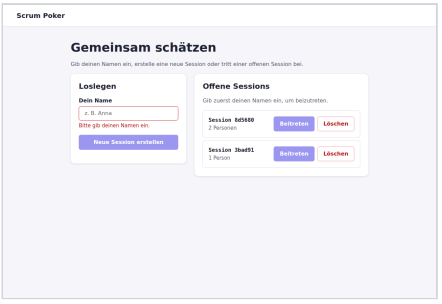
T-Shirt-Deck nach dem Aufdecken

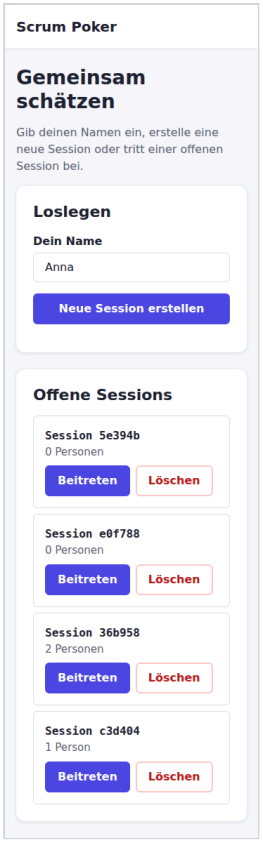
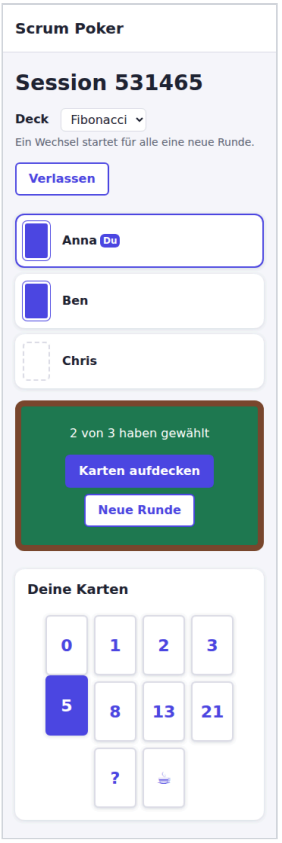


Session gelöscht



Ungültiger Name



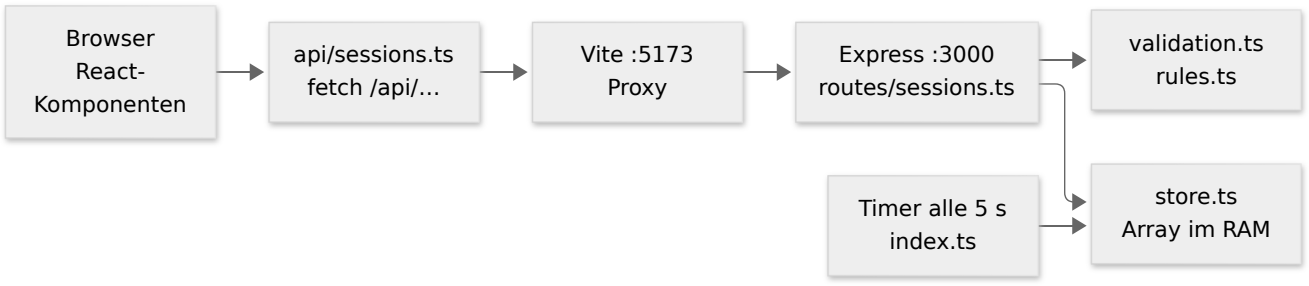
Startseite mobil	Pokertisch mobil
	

2. Architektur im Überblick

Das Projekt besteht aus **zwei getrennten Node-Projekten** nebeneinander, jedes mit eigener `package.json` (Folie 03/10: jede `package.json` steht für ein Projekt):

Ordner	Technik	Aufgabe
`client/`	React 19, TypeScript, Vite, oxlint, Prettier	Oberfläche im Browser. Aufgebaut wie die **Projektvorlage «scrum-poker-hwz»** (Folie 07/4): gleiche Scripts, gleiche Konfiguration, Ordner `components/`, `hooks/`, `types/`
`server/`	Node.js 24, Express 5, TypeScript	REST-API unter `/api/sessions`, Daten nur im Arbeitsspeicher

Die Projektvorlage gilt nur für den Client. Den Server haben wir nach den Folien zu Node.js (Foliensatz 03) und zur Web-Architektur (Foliensatz 02) aufgebaut. Express selbst wird in den Folien 1–9 nur genannt (01/4, 02/4, 09/37), aber noch nicht erklärt; die Aufgabe verlangt es.



- **Schichten wie in Folie 02/4.** Der Browser zeigt die Oberfläche (Presentation), Express ist Web-Server und Anwendung in einem. Die beiden reden nur über HTTP und JSON miteinander.
- **Kein CORS nötig.** Der Client ruft nur relative Adressen wie `/api/sessions` auf. Der Vite-Dev-Server leitet sie an Port 3000 weiter (`client/vite.config.ts:10-12`, so steht es schon in der Vorlage). Für den Browser gibt es damit nur eine Adresse (`localhost:5173`).
- **Drei Schichten im Server.** Router (HTTP, Statuscodes) → Prüfung der Eingaben → Store (Daten und Regeln, ohne Express). Den Store kann man lesen, ohne Express zu kennen.
- **Daten nur im Arbeitsspeicher.** Ein Array `Session[]` (`server/src/store.ts:33`). Nach einem Neustart des Servers sind alle Sessions weg. Für eine Schätzrunde ist das in Ordnung, die Aufgabe verlangt es so.
- **TypeScript ohne Build im Server.** Node 24 kann `.ts`-Dateien direkt ausführen, es entfernt die Typen beim Laden («Type Stripping»). Darum startet der Server mit `node --watch src/index.ts` (`server/package.json:11-12`), ohne zusätzliches Werkzeug. `pnpm build` im Server macht nur die Typprüfung mit `tsc`.
- **Produktivbetrieb.** Mit `NODE_ENV=production` liefert Express zusätzlich den gebauten Client aus `client/dist` aus (`server/src/app.ts:24-26`). Dann braucht es Vite nicht, und App und API laufen unter derselben Adresse. Das passt zu Folie 02/6: Der Server liefert den Client-Code als Dateien aus.

Bewusst doppelt: Types und Regeln. Client und Server sind getrennte Projekte und importieren nichts voneinander. Die Types der API stehen darum zweimal: im Client in `client/src/types/poker.ts` (Ordner `types/` aus der Vorlage), im Server in `server/src/types.ts`. Ebenso die Kartenlisten und die Namensregel `validateName()`: im Client in `client/src/utils/poker.ts`, im Server in `server/src/rules.ts` (28 Zeilen). Jede dieser Dateien verweist im Kommentar oben auf ihr Gegenstück. Wer eine Regel ändert, muss es an beiden Stellen tun. Wir haben das in Kauf genommen, weil ein gemeinsames drittes Paket (ein pnpm-Workspace) nicht Kursstoff ist und die Vorlage keins vorsieht.

Bewusst einfach gehalten. Die Gruppe hat den Code zweimal überarbeiten lassen (Kapitel 15): zuerst vereinfacht, damit jedes Mitglied jede Stelle erklären kann, danach an die Kursfolien und die Projektvorlage angeglichen. Es gibt darum zum Beispiel keine Klassen, keinen React Router, keine Test-Libraries und keinen `AbortController`. Jede Seite, die pollt, hat ihren eigenen gut lesbaren `useEffect`, und die API-Funktionen werfen normale `Error`-Objekte mit deutscher Meldung.

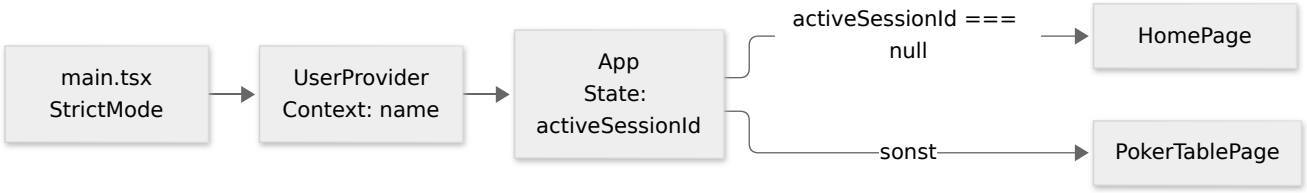
---

3. Komponentenstruktur und State

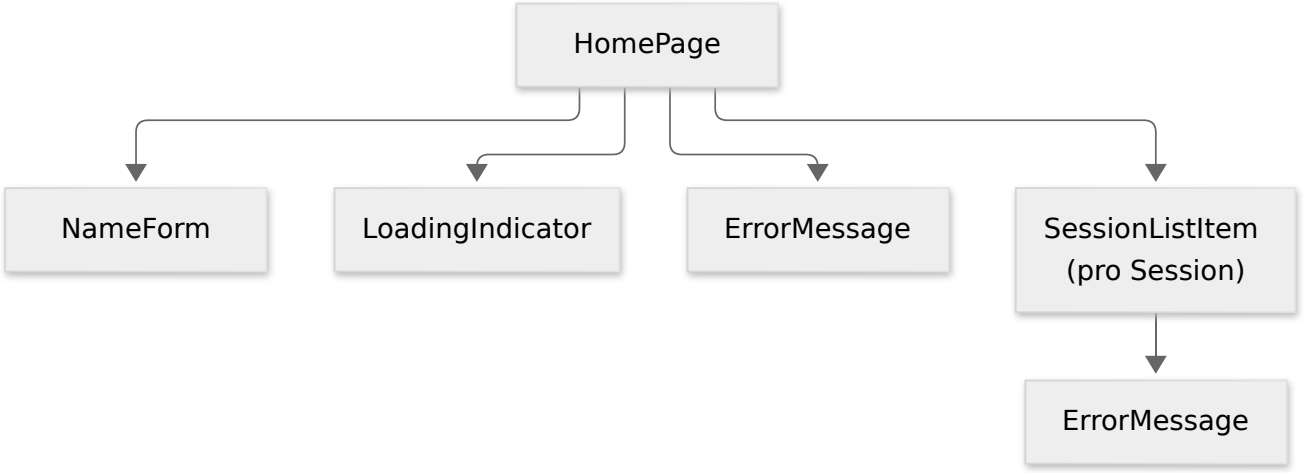
3.1 Komponentenbaum

Alle Komponenten liegen in `client/src/components/`, jede mit einer eigenen CSS-Datei daneben (Kapitel 11). Den Baum kann man im Browser auch mit den React Developer Tools ansehen (Folie 07/5).

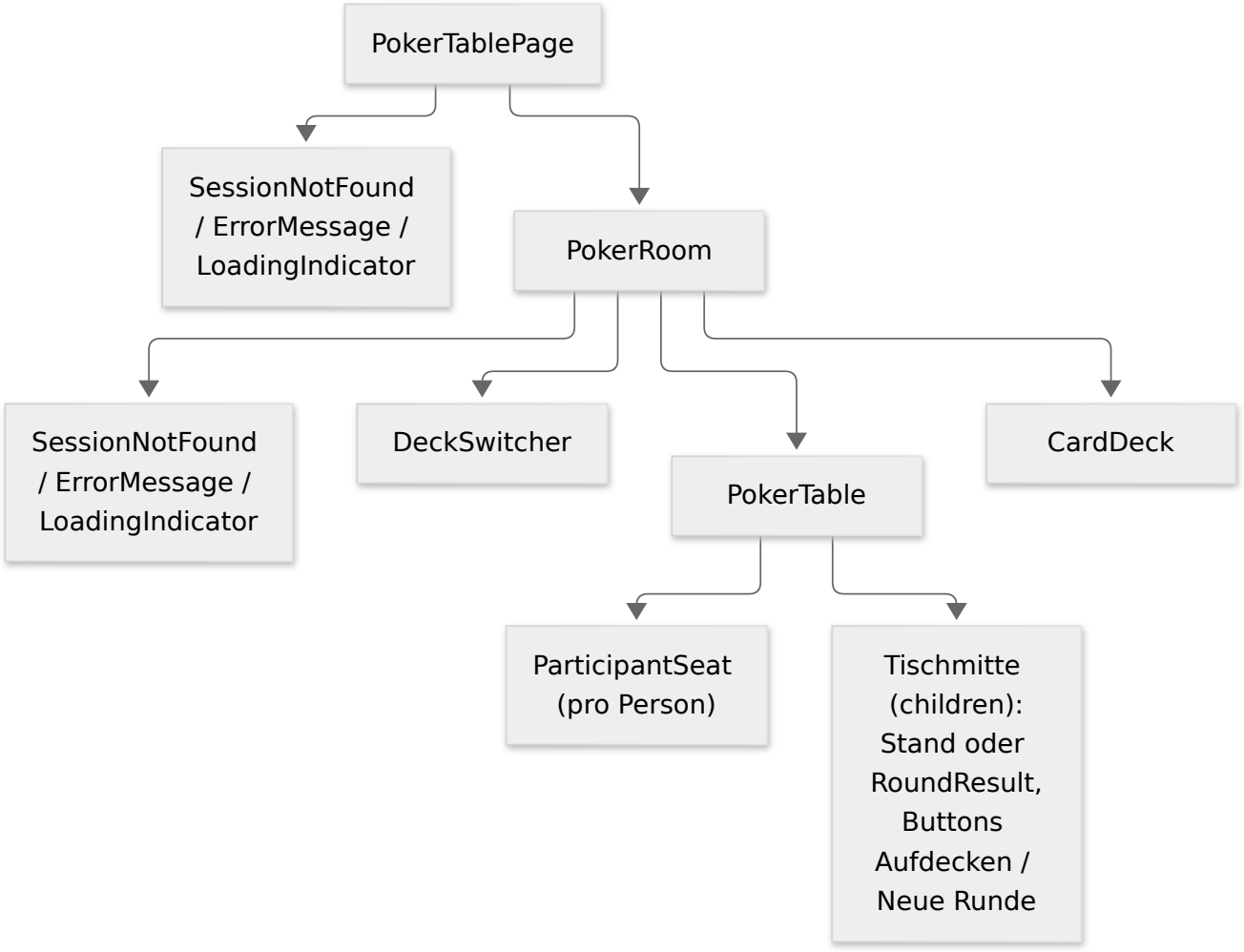
Gerüst und Ansichtswechsel



Startseite



Pokertisch



`PokerTablePage` kümmert sich ums Beitreten und Verlassen, `PokerRoom` um den Tisch selbst (Polling und Aktionen). So bleibt jede der beiden Dateien bei einer Aufgabe. `PokerTable` zeichnet nur die Plätze. Was in der Tischmitte steht, gibt `PokerRoom` als `children` mit (`PokerRoom.tsx:190-218`), wie in Folie 08/17. So muss `PokerTable` nichts vom Aufdecken oder vom Ergebnis wissen.

3.2 Komponenten, Props und wer sie rendert

Komponente	Gerendert von	Props	Eigener State
`App`	`main.tsx`	–	`activeSessionId`
`UserProvider`	`main.tsx`	`children`	`name` (über `useLocalStorage`)
`HomePage`	`App`	`onJoin` (liest den Namen aus dem Context)	`sessions`, `listError`, `isCreating`, `createError`
`NameForm`	`HomePage`	`name`, `error`, `onNameChange`	–
`SessionListItem`	`HomePage`	`session`, `canJoin`, `onJoin`, `onDeleted`	`isDeleting`, `deleteError`
`PokerTablePage`	`App`	`sessionId`, `onLeave` (liest den Namen aus dem Context)	`participantId`, `notFound`, `joinError`
`SessionNotFound`	`PokerTablePage`, `PokerRoom`	`onBack`	–
`PokerRoom`	`PokerTablePage`	`sessionId`, `participantId`, `onLeave`	`session`, `pollError`, `notFound`, `isBusy`, `actionError`
`DeckSwitcher`	`PokerRoom`	`deck`, `disabled`, `onChange`	–
`PokerTable`	`PokerRoom`	`participants`, `ownId`, `revealed`, `children`	–
`ParticipantSeat`	`PokerTable`	`participant`, `isOwn`, `revealed`	–
`RoundResult`	`PokerRoom` (in der Tischmitte)	`participants`, `deck`	– (wird berechnet)
`CardDeck`	`PokerRoom`	`deck`, `selected`, `disabled`, `onCardClick`	–
`LoadingIndicator`	`HomePage`, `PokerTablePage`, `PokerRoom`	`label`	–
`ErrorMessage`	`HomePage`, `SessionListItem`, `PokerTablePage`, `PokerRoom`	`message`	–

Die Props sind überall gleich geschrieben: ein `type Props = { … }` direkt über der Komponente und Destructuring in der Parameterliste, so wie in Folie 08/15 und 07/20. Beispiel: `client/src/components/CardDeck.tsx:7-15`. Wir verwenden durchgehend `type` und nie `interface`, weil die Folien das empfehlen (06/22) und das Projekt einheitlich sein soll (09/37). Keine Komponente hat mehr als vier Props (Folie 08/48 nennt weniger als acht als Richtwert). Alle Komponenten sind Funktionskomponenten mit Named Export (`export function CardDeck`), wie `App` in der Vorlage; Klassen gibt es keine (Folie 08/14 und 09/37). Jede Datei beginnt mit einem kurzen deutschen Kommentar: wozu sie da ist und wer sie benutzt.

3.3 Wo liegt welcher State – und warum?

State	Ort	Begründung
`activeSessionId`	`App` (`App.tsx:11`)	Entscheidet, welche Seite zu sehen ist (Kapitel 4). Liegt in `App`, weil beide Seiten ihn ändern: `HomePage` setzt ihn, der Pokertisch setzt ihn zurück.
Name	`UserProvider` mit `useLocalStorage` (`UserProvider.tsx:13`)	Wird auf der Startseite und am Pokertisch gebraucht. Darum liegt er ganz oben, um die `App` herum.
Gültigkeit des Namens	nirgends, berechnet mit `validateName(name)` (`HomePage.tsx:30-32`)	Abgeleiteter Wert (Folie 08/28). Gespeichert könnte er veralten.
Liste der Sessions, `listError`	`HomePage` (`HomePage.tsx:35-36`)	Nur die Startseite braucht sie. `SessionListItem` bekommt eine Session über Props.
`isCreating`, `createError`	`HomePage` (`HomePage.tsx:37-38`)	Gehört zum Button «Neue Session erstellen», der direkt in der Seite steht.
`isDeleting`, `deleteError`	`SessionListItem` (`SessionListItem.tsx:26-27`)	Jeder Eintrag hat seinen eigenen Ladezustand beim Löschen.
`participantId`, `notFound`, `joinError`	`PokerTablePage` (`PokerTablePage.tsx:23-25`)	Gehört zum Beitreten und Verlassen, das hier im `useEffect` passiert.
Session-Zustand (Teilnehmer, Deck, `revealed`, eigene Stimme)	`PokerRoom` (`PokerRoom.tsx:36`)	**Massgebend ist immer der Zustand auf dem Server.** Eigene Karte, Deck und «aufgedeckt» werden daraus gelesen und nicht doppelt gespeichert.
`isBusy`, `actionError`, `pollError`, `notFound`	`PokerRoom` (`PokerRoom.tsx:37-41`)	Ladezustand und Fehler aller Aktionen am Tisch an einer Stelle.
Ergebnis, «x von y haben gewählt»	nirgends, bei jedem Render berechnet	Reine Ableitung aus dem Session-Zustand (`RoundResult.tsx:15-25`, `PokerRoom.tsx:127-131`).

Eine Quelle pro Information. Jede Information liegt an genau einer Stelle. Alles andere wird bei jedem Render daraus berechnet, wie in den Tipps zu State in Folie 08/28 (Summe, Durchschnitt, Anzahl ableiten statt speichern). Ein Beispiel: Die eigene Karte könnte man nach dem Klick in einem eigenen `useState` merken. Dann gäbe es aber zwei Wahrheiten. Klickt eine andere Person auf «Neue Runde», bliebe die Karte bei uns hervorgehoben, obwohl der Server die Stimme schon gelöscht hat. Darum liest `PokerRoom` die eigene Stimme aus der Server-Antwort (`me.vote`, `PokerRoom.tsx:108-127`). Der Server schickt dafür die eigene Stimme schon vor dem Aufdecken mit, die der anderen nicht (`server/src/store.ts:45-48`).

Lifting State Up und State Flow (Folie 08/25-27). Die Komponenten unter `PokerRoom` (`CardDeck`, `DeckSwitcher`, `PokerTable`, `ParticipantSeat`, `RoundResult`) haben keinen eigenen State. Sie bekommen Daten über Props nach unten und melden Klicks über Callbacks nach oben (`onCardClick`, `onChange`). Der State liegt im ersten gemeinsamen Parent, der ihn braucht: `PokerRoom`. Genauso auf der Startseite: `NameForm` ist ein kontrolliertes Eingabefeld (`value={name}`, `onChange` → `onNameChange`), der Wert liegt im Context. Und ganz oben: `App` hält `activeSessionId`, die Seiten ändern ihn nur über die Callbacks `onJoin` und `onLeave`.

Context für den Namen (nicht in den Folien 1–9). Die Aufgabenstellung verlangt für den Namen die Context API. Den Namen braucht die Startseite (Eingabe) und der Pokertisch (Beitreten). Ohne Context müsste `App` den Namen halten und an beide Seiten weitergeben. Mit Context liest jede

Komponente ihn direkt: `const { name, setName } = useContext(UserContext)` (`HomePage.tsx:30`, `PokerTablePage.tsx:21`). Der Context selbst steht in `client/src/context/UserContext.ts`, der Provider, der ihn füllt, in `client/src/components/UserProvider.tsx`. Seit React 19 ist der Context selbst der Provider, man schreibt also `<UserContext value={…}>` (`UserProvider.tsx:17-21`).

Wann sind Props besser? Bei allem anderen in dieser App:

- Daten, die nur eine Ebene nach unten gehen (z. B. eine `session` an `SessionListItem`). Props zeigen, woher die Daten kommen.
- Daten, die sich oft ändern, wie der Session-Zustand (alle 1.5 s). Ändert sich ein Context, rendern **alle** Komponenten neu, die ihn lesen. Bei Props rendert nur der betroffene Teil.

Context lohnt sich für wenige, selten geänderte Werte, die viele Stellen auf verschiedenen Ebenen brauchen (Name, Sprache, Farbschema, eingeloggte Person).

3.4 Beitreten, Verlassen und React StrictMode

Beitreten und Verlassen passieren an genau einer Stelle: im Effekt von `PokerTablePage` (`PokerTablePage.tsx:30-71`). Das ist das Mount/Unmount-Muster aus Folie 09/13 und 09/15: Code beim Mounten im Effekt, Code beim Unmounten im Cleanup.

- **Beitreten** beim Mount mit `joinSession()`. Das Dependency Array `[sessionId, name]` sorgt dafür, dass bei einer anderen Session zuerst verlassen und dann neu beigetreten würde. In der App kommt das nicht vor, weil man den Tisch dafür verlassen muss.
- **Verlassen** im Cleanup (`PokerTablePage.tsx:63-70`). Der Button «Verlassen» ruft nur `onLeave` auf. `App` setzt `activeSessionId` auf `null` und zeigt wieder die Startseite. `PokerTablePage` verschwindet, und ihr Cleanup schickt den Leave-Request. Es gibt also genau einen Leave-Request, egal über welchen Button man den Tisch verlässt («Verlassen», «Zurück zur Übersicht»).
- **Reload oder Tab schliessen:** Dabei läuft kein Cleanup, es wird nichts gesendet. Der Heartbeat-Timeout entfernt die Person (Kapitel 8). Nach einem Reload steht man auf der Startseite mit vorbelegtem Namen und kann neu beitreten.
- **StrictMode** (`main.tsx:16-22`) startet in der Entwicklung jeden Effekt beim ersten Anzeigen absichtlich zweimal: Effekt → Cleanup → Effekt (Folie 08/24 und 09/3). Dadurch gehen zwei Join-Requests los. Das **`ignore`-Flag** (`PokerTablePage.tsx:34`) ist das Muster aus der React-Dokumentation: Das Cleanup setzt `ignore = true`, und die Antwort des ersten, schon aufgeräumten Effekts wird nicht mehr verwendet. Den Teilnehmer, den dieser erste Join angelegt hat, meldet der Code sofort wieder ab (`PokerTablePage.tsx:40-47`). Es bleibt also genau ein Teilnehmer pro Fenster. Im gebauten Client (`pnpm build`) läuft der Effekt nur einmal.
- **Vom Server entfernt** (z. B. Laptop im Ruhemodus): Fehlt die eigene `participantId` in der Teilnehmerliste, zeigt `PokerRoom` «Du bist nicht mehr am Tisch» mit dem Button «Zurück zur Übersicht» (`PokerRoom.tsx:111-125`).

---

4. Ansichtswechsel mit State

Die App hat zwei Ansichten: die Startseite (`HomePage`) und den Pokertisch (`PokerTablePage`). Gewechselt wird ohne Router, nur mit State in `App`. Das ist die Pflicht §3.1 aus dem geänderten Leistungsnachweis (Folie 07/6) und genau so in Folie 09/36 beschrieben.

```
// client/src/components/App.tsx
const [activeSessionId, setActiveSessionId] = useState<string | null>(null)
```

```
function leaveSession() {
  setActiveSessionId(null)
}

let page = <HomePage onJoin={setActiveSessionId} />
if (activeSessionId !== null) {
  page = <PokerTablePage sessionId={activeSessionId} onLeave={leaveSession} />
}
```

(`App.tsx:11-20`)

- **State:** `activeSessionId` ist `null` auf der Startseite und sonst die ID der Session, an deren Tisch man sitzt. Der Typ `string | null` ist ein Union Type mit `null` (Folie 06/14); das Generic `useState<string | null>` braucht es, weil TypeScript aus dem Startwert `null` allein den Typ nicht kennt (Folie 07/16).
- **Conditional Rendering** (Folie 08/37): Ist eine ID gesetzt, rendert `App` die `PokerTablePage`, sonst die `HomePage`. Der Header bleibt darum auf beiden Ansichten gleich (`App.tsx:22-29`).
- **Session-ID per Props:** `PokerTablePage` bekommt die ID als Prop `sessionId` und gibt sie an `PokerRoom` weiter.
- **Callbacks via Props** (Folie 08/26):

Wo	Wann	Was passiert
`HomePage.tsx:82`	nach erfolgreichem `POST /api/sessions`	`onJoin(sessionId)` → Tisch der neuen Session
`SessionListItem.tsx:69`	Klick auf «Beitreten»	`onJoin(session.id)` → Tisch dieser Session
`PokerRoom.tsx:176-182`	Klick auf «Verlassen»	`onLeave()` → `App` setzt den State auf `null` → Startseite
`SessionNotFound.tsx:14-20`	Session gibt es nicht (mehr)	Button «Zurück zur Übersicht» ruft `onBack` (= `onLeave`)

`HomePage` gibt `onJoin` unverändert an jeden `SessionListItem` weiter. Der Setter `setActiveSessionId` ist selbst eine Funktion `(id: string) => void` und passt darum direkt als Callback (Funktionen als Parameter, Folie 06/31). «Beitreten» auf der Startseite wechselt nur die Ansicht. Das eigentliche Beitreten (`POST …/participants`) macht `PokerTablePage` beim Mount.

Warum kein React Router? Der Router ist laut Folie 07/6 nur noch eine optionale Erweiterung (§3.4). Eine frühere Fassung der App hatte ihn, mit Adressen wie `/sessions/abc123`, einem Button «Link kopieren» und Fortsetzen nach einem Reload. Mit dem geänderten Leistungsnachweis braucht es das alles nicht mehr. Wir haben den Router entfernt, damit es für den Ansichtswechsel nur einen Weg gibt. Folge: Die Adresse im Browser bleibt immer `/`, und der Zurück-Button des Browsers verlässt die App, statt zur Startseite zu wechseln.

---

5. REST-API

Basis-Adresse: `http://localhost:3000/api` (im Browser über den Proxy `http://localhost:5173/api`). Alle Bodies sind JSON. Jede Fehlerantwort hat die Form `{ "error": "…" }`. Die Routen stehen in `server/src/routes/sessions.ts`, die Types der Bodies in `server/src/types.ts` und gleich im Client in `client/src/types/poker.ts:43-69`.

Methode	Pfad	Body	Antwort	Statuscodes
`GET`	`/api/sessions`	–	`SessionSummary[]`	200

Methode	Pfad	Body	Antwort	Statuscodes
`POST`	`/api/sessions`	–	`{ id }`	201
`DELETE`	`/api/sessions/:sessionId`	–	–	204, 404
`GET`	`/api/sessions/:sessionId?participantId=…`	–	`SessionState`	200, 404
`POST`	`/api/sessions/:sessionId/participants`	`{ name }`	`{ participantId }`	201, 400, 404
`DELETE`	`/api/sessions/:sessionId/participants/:participantId`	–	–	204, 404
`PUT`	`/api/sessions/:sessionId/participants/:participantId/vote`	`{ value: CardValue | null }`	`SessionState`	200, 400, 404, 409
`PUT`	`/api/sessions/:sessionId/deck`	`{ deck: 'fibonacci' | 'tshirt' }`	`SessionState`	200, 400, 404
`POST`	`/api/sessions/:sessionId/reveal`	–	`SessionState`	200, 404
`POST`	`/api/sessions/:sessionId/reset`	–	`SessionState`	200, 404

Für alle Routen gilt zusätzlich: ungültiges JSON → 400, unbekannte Route → 404, unerwarteter Fehler → 500, immer als JSON und nie mit Stacktrace (`server/src/app.ts:29-43`).

Jede Route ist gleich aufgebaut und liest sich von oben nach unten: Session holen → gibt es sie nicht, 404 → Body prüfen (400) → Store ändern → mit Statuscode und JSON antworten. Beispiel: `PUT …/vote` in `routes/sessions.ts:116-145`.

Begründung der HTTP-Methoden

Die Methoden sind die aus Folie 02/11 (HTTP 1.1).

Methode	Verwendet für	Warum
`GET`	Liste, Session-Zustand	Liest nur und darf beliebig oft wiederholt werden. Einzige Nebenwirkung: Mit `participantId` merkt sich der Server «diese Person ist noch da» (`lastSeen`, Heartbeat). Das ist Verwaltung, keine fachliche Änderung.
`POST`	Session erstellen, beitreten	Erstellt etwas Neues, dessen ID der **Server** vergibt → `201 Created`. Zweimal senden erzeugt zwei Einträge.
`POST`	`reveal`, `reset`	Das sind Aktionen («aufdecken», «neue Runde»), kein Wert, den der Client vollständig setzt.
`PUT`	`vote`, `deck`	Der Client setzt einen Wert vollständig neu. **Idempotent:** Derselbe Request zweimal ergibt denselben Zustand. `value: null` zieht die Stimme zurück.
`DELETE`	Session, Teilnehmer	Entfernt etwas → `204 No Content`. Teilnehmer entfernen ist idempotent: Auch ein schon entfernter Teilnehmer ergibt 204, denn das Ziel («nicht mehr dabei») ist erreicht.

Statuscodes

Code	Bedeutung in dieser API
200	OK, die Antwort enthält Daten
201	Session oder Teilnehmer neu erstellt
204	Gelöscht, kein Body

Code	Bedeutung in dieser API
400	Body ungültig: kein JSON, Name leer oder über 30 Zeichen, Karte nicht im aktiven Deck, unbekanntes Deck
404	Session unbekannt, Teilnehmer unbekannt (beim Abstimmen) oder Route unbekannt
409	Abstimmen, nachdem die Karten aufgedeckt sind (passt nicht zum aktuellen Zustand)
500	Unerwarteter Serverfehler

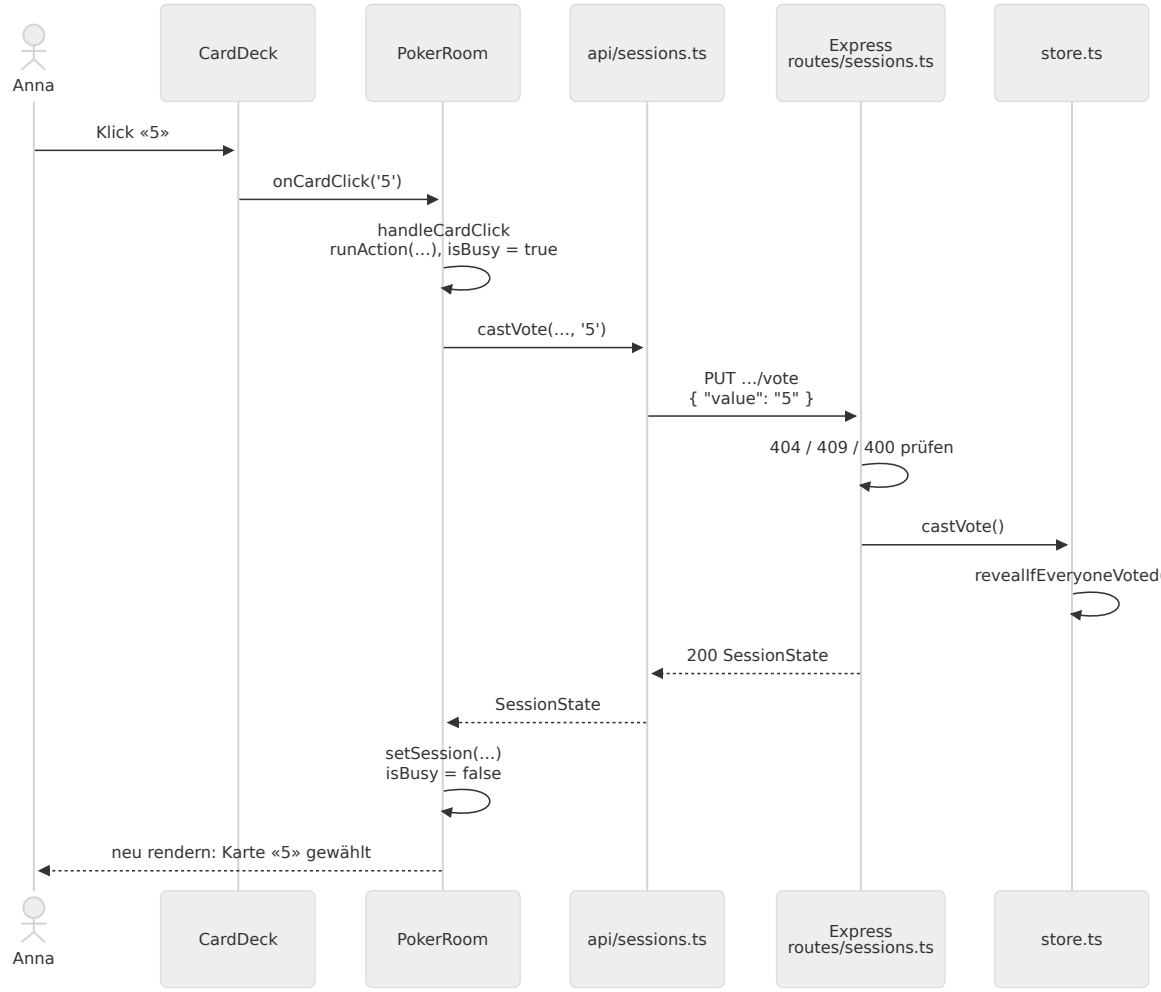
Fachliche Regeln im Server

- **Stimmen bleiben geheim:** Vor dem Aufdecken schickt der Server pro Person nur `hasVoted`, `vote` ist `null`. Ausnahme ist die eigene Stimme der fragenden Person (`store.ts:39-62`). Die Karte der anderen verlässt den Server also nie zu früh, auch nicht im Network-Tab der Entwicklertools (Folie 02/19).
- **Automatisch aufdecken:** Haben alle (mindestens eine Person) gewählt, deckt der Server auf. Das gilt auch, wenn die letzte Person ohne Stimme geht (`store.ts:126-135`, `store.ts:165-175`).
- **Deck-Wechsel** startet eine neue Runde, weil alte Karten im neuen Deck ungültig wären. Dasselbe Deck nochmals wählen ändert nichts (`store.ts:188-195`).
- **Unbekannte `participantId` beim GET** ist kein Fehler (200). Der Client sieht an der Teilnehmerliste selbst, dass er nicht mehr dabei ist, und zeigt «Du bist nicht mehr am Tisch».
- **Eingaben werden immer geprüft**, auch wenn der Client schon prüft. Ein Request kann auch von `curl` kommen. Der Body ist darum `unknown` und wird Schritt für Schritt mit `typeof` und `in` geprüft (`server/src/validation.ts`). `unknown` und `in` sind nicht in den Folien; mit `any` würde TypeScript aber gar nichts prüfen. Erst nach der Prüfung weist die Route den Body einer Variable mit unserem Type zu (`const joinRequest: JoinSessionRequest = req.body`, `routes/sessions.ts:95-96`).
- **Kurze Session-IDs:** 6 Zeichen aus `randomUUID()` (Node-Modul `crypto`, nicht in den Folien), damit man sie in der Liste gut lesen und vorlesen kann. Ist eine ID schon vergeben, wird neu gewürfelt (`store.ts:78-81`).
- **Reihenfolge der Liste:** Das Array wächst mit `push`, die Liste kommt also in Erstellungsreihenfolge, die älteste Session zuerst (`store.ts:64-73`).

---

6. Datenfluss: Klick auf eine Karte

Beispiel: Anna klickt im Fibonacci-Deck auf «5». Ben sitzt am selben Tisch und hat noch nicht gewählt.



Schritt für Schritt am Code:

1. **Klick.** Jede Karte ist ein `<button>` mit `onClick={() => onCardClick(value)}` (`CardDeck.tsx:22-34`). Die Karten entstehen mit `map` aus der Kartenliste des Decks, jede mit `key={value}` (Folie 08/30-31). `CardDeck` kennt nur die Karte, die Entscheidung trifft `PokerRoom` (Lifting State Up).
2. **Entscheidung.** `handleCardClick` vergleicht mit der eigenen Stimme aus dem Server-Zustand: Ist «5» schon gewählt, wird `null` gesendet (zurückziehen), sonst `'5'` (`PokerRoom.tsx:145-152`).
3. **Ladezustand.** `runAction` bekommt die Aktion als Funktion übergeben (`() => castVote(…)`, Funktionen als Parameter wie in Folie 06/31), setzt `isBusy` auf `true` und löscht einen alten Fehler (`PokerRoom.tsx:85-95`). Solange `isBusy` gilt, sind alle Karten und Buttons gesperrt (`disabled={session.revealed || isBusy}`, `PokerRoom.tsx:223`), und in der Tischmitte steht «Wird gesendet …». Ein Doppelklick schickt so keinen zweiten Request.
4. **HTTP-Request.** `castVote()` baut den Body vom Typ `CastVoteRequest` und schickt `PUT /api/sessions/:id/participants/:pid/vote` (`api/sessions.ts:155-171`). Die Hilfsfunktion `send()` setzt den Header `Content-Type: application/json` und macht aus «keine Antwort» die Meldung «Server nicht erreichbar.» (`api/sessions.ts:21-38`). Wie in Folie 09/22-23: `await fetch(…)`, dann `response.ok` prüfen und `await response.json()`.
5. **Proxy.** Vite leitet `/api/…` an `localhost:3000` weiter (`vite.config.ts:10-12`).
6. **Server.** `express.json()` liest den Body (`app.ts:19`). Die Route (`routes/sessions.ts:116-145`) prüft der Reihe nach: Session vorhanden (sonst 404), Teilnehmer vorhanden (sonst

404), noch nicht aufgedeckt (sonst 409), Karte im aktiven Deck (sonst 400, `validation.ts:22-37`).

7. **Store.** `castVote()` speichert die Stimme und ruft `revealIfEveryoneVoted()` auf (`store.ts:155-175`). Hätte Ben schon gewählt, wäre jetzt `revealed = true`.
8. **Antwort.** Die Route schickt `toSessionState(session, participant.id)` (`routes/sessions.ts:143`). Annas eigener Eintrag enthält `vote: "5"`, alle anderen `vote: null` (`store.ts:45-48`).
9. **Neuer State.** `runAction` legt die Antwort mit `setSession(newSession)` ab und setzt danach `isBusy` zurück (`PokerRoom.tsx:89-94`). Die Antwort wird sofort angezeigt, die App wartet nicht auf den nächsten Poll.
10. **Anzeige.** React rendert `PokerRoom` neu. `myVote` ist jetzt `'5'` (`PokerRoom.tsx:127`), `CardDeck` bekommt `selected="5"`, und die Karte erhält die Klasse `playing-card--selected` (Conditional Styling, Folie 08/36, `CardDeck.tsx:25-29`). Annas Platz zeigt eine verdeckte Karte (`ParticipantSeat.tsx:16-18`), und in der Tischmitte steht «1 von 2 haben gewählt» (`PokerRoom.tsx:134-138`).
11. **Andere Browser.** Ben sieht die verdeckte Karte bei seiner nächsten Abfrage (spätestens nach 1.5 s), weil sein `GET` jetzt `hasVoted: true` für Anna liefert.

---

7. Lade- und Fehlerzustände

Alle Requests laufen über `client/src/api/sessions.ts`. Die Komponenten rufen nie selbst `fetch()` auf. Jede API-Funktion wirft bei einem Fehler ein normales `Error` mit einer deutschen Meldung (`throw new Error(…)` kommt in den Folien nicht vor, ist aber der einfachste Weg, einen Fehler mit Text an das `catch` der Komponente zu geben). «Diese Session gibt es nicht» (404) ist dagegen **kein** Fehler, sondern der Rückgabewert `null`. Das ist leichter zu lesen als eine Prüfung auf einen Statuscode.

Ursache	Ergebnis der API-Funktion	Stelle
Server läuft nicht, kein Netz (`fetch` wirft)	Fehler «Server nicht erreichbar.»	`api/sessions.ts:35-37`
Vite-Proxy erreicht Express nicht (502/503/504)	Fehler «Server nicht erreichbar.»	`api/sessions.ts:43-49`
Antwort mit `{ error }` (400, 404 bei Aktionen, 409, 500)	Fehler mit dem Text vom Server	`api/sessions.ts:50-55`
Fehlerantwort ohne JSON	Fehler «Anfrage fehlgeschlagen (HTTP …).»	`api/sessions.ts:59`
404 bei `getSession` oder `joinSession`	`null` («gibt es nicht»)	`api/sessions.ts:108-110`, `api/sessions.ts:130-132`
404 bei `deleteSession`	kein Fehler (schon gelöscht, Ziel erreicht)	`api/sessions.ts:89-91`

Im `catch` ist der Fehler vom Typ `unknown`. `getErrorMessage(error)` prüft mit `instanceof`, ob es wirklich ein `Error` ist, und holt daraus den Text für die Anzeige (`api/sessions.ts:213-218`).

So behandelt jede Stelle der App die Anfragen:

Situation	Was man sieht	Code
Liste lädt das erste Mal	`LoadingIndicator` «Sessions werden geladen …»	`HomePage.tsx:140`

Situation	Was man sieht	Code
Server beim Laden der Liste nicht erreichbar	`ErrorMessage` «… Neuer Versuch läuft automatisch.» Die letzte Liste bleibt sichtbar, das Polling versucht es weiter.	`HomePage.tsx:55-60`, `HomePage.tsx:134-138`
Session erstellen läuft	Button gesperrt mit Text «Wird erstellt …»	`HomePage.tsx:120-127`
Session erstellen fehlgeschlagen	Meldung unter dem Button, Button wieder aktiv	`HomePage.tsx:83-88`, `HomePage.tsx:128`
Löschen läuft	Button gesperrt mit Text «Wird gelöscht …»	`SessionListItem.tsx:73-80`
Löschen fehlgeschlagen	Meldung im Listeneintrag	`SessionListItem.tsx:52-55`, `SessionListItem.tsx:82`
Beitreten läuft	`LoadingIndicator` «Beitritt zur Session …»	`PokerTablePage.tsx:91-93`
Unbekannte Session beim Beitreten	`SessionNotFound` mit Button «Zurück zur Übersicht»	`PokerTablePage.tsx:48-51`, `PokerTablePage.tsx:74-76`
Anderer Fehler beim Beitreten (z. B. Server weg)	`ErrorMessage` und Button «Zurück zur Übersicht»	`PokerTablePage.tsx:77-90`
Session wird gelöscht, während man am Tisch sitzt	dieselbe Meldung `SessionNotFound`. Das Polling hört auf.	`PokerRoom.tsx:53-58`, `PokerRoom.tsx:97-99`
Erste Abfrage am Tisch läuft oder schlägt fehl	`LoadingIndicator` «Tisch wird geladen …» bzw. `ErrorMessage`	`PokerRoom.tsx:100-106`
Server antwortet beim Polling nicht	`ErrorMessage` «… Neuer Versuch läuft …». Der letzte Stand des Tisches bleibt sichtbar.	`PokerRoom.tsx:62-67`, `PokerRoom.tsx:185-187`
Vom Server entfernt (Timeout)	«Du bist nicht mehr am Tisch» + «Zurück zur Übersicht»	`PokerRoom.tsx:111-125`
Aktion läuft (Karte, Deck, Aufdecken, Neue Runde)	«Wird gesendet …» in der Tischmitte, Karten, Auswahlfeld und Buttons gesperrt und ausgegraut	`PokerRoom.tsx:197`, `PokerRoom.tsx:199-216`, `PokerRoom.tsx:220-225`
Aktion fehlgeschlagen (z. B. 409 nach dem Aufdecken)	`ErrorMessage`, der Tisch bleibt bedienbar	`PokerRoom.tsx:91-93`, `PokerRoom.tsx:188`
Verlassen	Die Ansicht wechselt sofort zur Startseite, der Leave-Request läuft im Hintergrund. Schlägt er fehl, entfernt der Heartbeat-Timeout die Person, darum wird der Fehler bewusst ignoriert.	`PokerTablePage.tsx:63-70`
Ungültiger Name (leer, nur Leerzeichen, zu lang)	Meldung unter dem Feld, rot umrandetes Feld, Buttons gesperrt	`NameForm.tsx:26-39`, `validateName`
Ungültige Eingabe, die den Client umgeht (`curl`)	Server antwortet 400 mit `{ error }`	`validation.ts`

Keine unbehandelten Promises. Jede `async`-Funktion fängt ihre Fehler mit `try`/`catch` selbst ab (`loadSessions`, `handleCreate`, `handleDelete`, `join`, `loadSession`, `runAction`), so wie in Folie 09/17. Der Leave-Request im Cleanup hat ein `.catch(() => {})`, weil sein Fehler bewusst egal ist. Darum kann ein Event-Handler eine `async`-Funktion einfach aufrufen, z. B. `onClick={handleCreate}`, ohne dass je eine «Uncaught (in promise)»-Meldung in der Konsole erscheint.

Keine Error Boundary. React bietet Error Boundaries nur als Klasse an, und die Folien schliessen Class-Components aus (08/14, 09/37). Eine frühere Fassung hatte eine; wir haben sie entfernt. Fehler von Server-Aufrufen fängt ohnehin das `try`/`catch` oben ab, eine Error Boundary würde nur Programmierfehler beim Rendern abfangen. Gegen solche Fehler helfen `strict`-TypeScript, `pnpm lint` und das Durchklicken vor der Abgabe (Kapitel 12). Passiert trotzdem einer, wird die Seite leer, und die Konsole zeigt den Fehler.

Späte Antworten. Alle drei Effekte (Polling der Startseite, Beitreten, Polling am Tisch) haben ein `ignore`-Flag. Das Cleanup setzt es auf `true`. Eine Antwort, die erst danach ankommt, wird nicht mehr in den State geschrieben (`HomePage.tsx:44-74`, `PokerTablePage.tsx:30-71`, `PokerRoom.tsx:44-80`). Einen `AbortController` (Request wirklich abbrechen) verwenden wir nicht, er ist kein Kursstoff. Die einzige Folge: Kommt eine alte Poll-Antwort zufällig erst nach der Antwort einer eigenen Aktion an, zeigt der Tisch für höchstens 1.5 s den alten Stand. Der nächste Poll korrigiert das. Lokal ist das praktisch nie zu sehen.

---

8. Strategie gegen Karteileichen

Problem. Schliesst jemand den Tab, verliert das WLAN oder klappt den Laptop zu, bleibt die Person als «Karteileiche» in der Session. Sie stimmt nie ab, also deckt der Server auch nie automatisch auf.

Die App arbeitet mit **zwei Schichten**: Wer den Tisch in der App verlässt, ist sofort weg. Wer einfach verschwindet (Tab zu, Reload, Absturz, Netz weg), wird vom Heartbeat-Timeout entfernt.

Schicht 1: sofort verlassen (Button in der App)

- «Verlassen» wechselt die Ansicht zur Startseite. `PokerTablePage` verschwindet, und das Cleanup ihres Effekts sendet `DELETE …/participants/:id` (`PokerTablePage.tsx:63-70`). Dasselbe passiert mit «Zurück zur Übersicht».
- Weil nur das Cleanup verlässt, gibt es genau **einen** Leave-Request, egal über welchen Button man geht.

Schicht 2: Heartbeat über das Polling, 15 s Timeout, Aufräumen alle 5 s

Beim Schliessen des Tabs, bei einem Reload, einem Absturz, leerem Akku, Netzverlust oder im Ruhemodus läuft kein React-Cleanup, und es wird nichts mehr gesendet. Dafür gibt es den Heartbeat:

1. Der Tisch fragt den Session-Zustand ohnehin alle **1.5 s** ab (`POLL_INTERVAL_MS`, `PokerRoom.tsx:25`) und schickt dabei seine `participantId` mit (`api/sessions.ts:104-107`). Das ist das Polling-Muster aus Folie 09/30: `window.setInterval` im `useEffect` starten, im Cleanup mit `window.clearInterval` stoppen.
2. Der Server wertet jede solche Abfrage als Lebenszeichen und setzt `lastSeen` auf jetzt (`routes/sessions.ts:74-79`).
3. Ein Timer läuft alle **5 s** (`server/src/index.ts:9`, `CLEANUP_INTERVAL_MS` in `server/src/config.ts:12`) und entfernt alle, deren `lastSeen` älter als **15 s** ist (`PARTICIPANT_TIMEOUT_MS`, `config.ts:16`; `store.ts:142-152`).

4. Das Entfernen läuft über `removeParticipant()`. Darum prüft es auch das automatische Aufdecken: War die Karteileiche die Letzte ohne Stimme, wird jetzt aufgedeckt (`store.ts:126-135`).
5. Merkt ein Client, dass er selbst nicht mehr in der Teilnehmerliste steht (z. B. nach dem Ruhemodus), zeigt er «Du bist nicht mehr am Tisch» und führt zurück zur Übersicht.

Warum diese Zahlen?

- 15 s sind zehn verpasste Abfragen. Ein kurzer Netzaussetzer oder ein langsamer Request wirft niemanden raus.
- Mit 5 s Prüfintervall ist eine verschwundene Person nach spätestens 15 + 5 = **20 s** weg. Das ist kurz genug, dass das Team nicht lange wartet.
- Beide Werte sind benannte Konstanten in `server/src/config.ts`, nicht im Code verstreut.

Reload. Nach einem Reload ist man auf der Startseite und kann wieder beitreten, dann als neue Person. Der alte Platz bekommt keine Polls mehr und verschwindet nach höchstens 20 s. Bis dahin sitzt man kurz doppelt am Tisch; von Hand aufdecken geht in dieser Zeit trotzdem. Mehr verlangt der Leistungsnachweis nicht (Folie 07/6: nach einem Reload bleibt nur der Name).

Verworfene Alternativen

Alternative	Warum verworfen
Nur ein Leave-Request beim Schliessen (ohne Heartbeat)	Unzuverlässig: Bei Absturz, leerem Akku, Netzverlust oder Ruhemodus wird nie etwas gesendet.
Zusätzlich Leave beim `pagehide`-Event (Tab schliessen)	`pagehide` ist kein Kursstoff, feuert auch beim Reload und kommt beim Absturz trotzdem nicht. Den Heartbeat bräuchte es also so oder so.
Eigener Heartbeat-Endpunkt (z. B. `POST …/heartbeat` alle 5 s)	Funktioniert, ist aber ein zusätzlicher Request und mehr Code. Das Polling findet sowieso statt und erfüllt denselben Zweck.
Kürzerer Timeout (z. B. 5 s)	Ein Tab im Hintergrund bremst seine Timer. Ein zu kurzer Timeout würde Personen entfernen, die nur kurz das Fenster gewechselt haben.
WebSockets: Verbindungsabbruch = weg	Erkennt Abbrüche sofort, braucht aber eine zweite Technik (WebSocket-Server, Wiederverbinden). Der Kurs setzt auf `fetch` und Polling (Folie 09/22-30).
Leere Sessions automatisch löschen	Wäre die letzte Person kurz weg (Reload, Netz), wäre die Session sonst verschwunden. Sessions werden nur von Hand gelöscht.

Bekannte Grenze: Chrome bremst die Timer eines Tabs, der länger als etwa fünf Minuten im Hintergrund liegt, auf einen Aufruf pro Minute. Dann fehlen die Polls, und der Server entfernt die Person nach 15 Sekunden. Kommt sie zurück, sieht sie «Du bist nicht mehr am Tisch» und tritt über die Liste neu bei. Für eine Schätzrunde, bei der alle den Tab offen haben, nehmen wir das in Kauf.

---

9. Durchschnitt bei T-Shirt-Grössen

T-Shirt-Grössen haben keinen Zahlenwert. Gewählte Variante: **jeder Grösse eine Zahl wie bei Fibonacci geben, Durchschnitt rechnen, dann die Grösse anzeigen, deren Zahl am nächsten liegt.**

Grösse	XS	S	M	L	XL
Wert	1	2	3	5	8

Die Tabelle steht als Liste `TSHIRT_SIZES` in `client/src/utils/poker.ts:32-38`: ein Array von Objekten `{ size, value }` vom Typ `TshirtSize`. Der Wert einer Grösse wird mit `find` gesucht (`utils/poker.ts:58-65`; `find` gibt `undefined`, wenn nichts passt, Folie 06/16). Beispiel: S (2) und XL (8) ergeben 5 → **L**. S (2) und L (5) ergeben 3.5 → am nächsten liegt M (3, Abstand 0.5) → **M**. Damit man das Ergebnis nachrechnen kann, steht darunter «Grössen als Zahlen (XS=1, S=2, M=3, L=5, XL=8): Durchschnitt 5, die nächste Grösse ist L» (`RoundResult.tsx:55-60`).

Begründung

- Die Werte folgen der Fibonacci-Folge wie das Zahlen-Deck. Die Abstände wachsen mit der Grösse, so wie die Unsicherheit bei grossen Aufgaben. Mit 1–5 in gleichen Abständen hätte ein XL zu wenig Gewicht.
- **Gleichstand** (z. B. 4 genau zwischen M=3 und L=5) → die **grössere** Grösse. Beim Schätzen ist Aufrunden die vorsichtigere Wahl. Im Code ist das das `<=` in `utils/poker.ts:81`: Bei gleichem Abstand gewinnt die später geprüfte, also grössere Grösse.
- `?` und `☕` haben keinen Zahlenwert und zählen nicht zum Durchschnitt, aber zur Anzahl Stimmen. `Number('?')` ergibt `NaN` (Folie 06/10), daran erkennt `toNumber()` diese Karten (mit `Number.isNaN`, nicht in den Folien). Gibt es gar keinen Zahlenwert, ist der Durchschnitt `null`, und es wird «–» angezeigt.
- Der Durchschnitt wird auf eine Nachkommastelle gerundet (4.333 → 4.3, `utils/poker.ts:119`).
- **Konsens** heisst: mindestens zwei Stimmen, und alle sind gleich (`utils/poker.ts:91-96`).

Verworfene Varianten: den Median oder die häufigste Grösse anzeigen (man sieht nicht mehr, wie weit die Schätzungen auseinanderliegen) oder gar keinen Durchschnitt (die Aufgabe verlangt einen).

Die Berechnung ist die Funktion `computeResult()` (`utils/poker.ts:89-125`). Sie läuft im Client, weil nur die Anzeige sie braucht. `RoundResult` ruft sie bei jedem Render mit den aufgedeckten Stimmen auf; das Ergebnis wird nicht gespeichert, sondern abgeleitet (Folie 08/28). Die Funktion hängt nicht von React ab: gleiche Stimmen ergeben immer dasselbe Ergebnis.

---

10. Browser-Storage

Die App speichert nur eines im Browser: den **Namen** im `localStorage` (Pflicht §3.2 laut Folie 09/36). Ihn sieht man in den Entwicklertools unter «Application» → «Local Storage» (Folie 02/19).

Speicher	`localStorage` (bleibt nach dem Schliessen des Browsers erhalten)
Key	`scrum-poker:name` (`UserProvider.tsx:13`)
Wert	der eingegebene Name, so wie er im Feld steht
Zweck	Der Name ist nach einem Reload und beim nächsten Besuch schon ausgefüllt.

Gelesen und geschrieben wird er im **Custom Hook** `useLocalStorage` (`client/src/hooks/useLocalStorage.ts`), so wie in der Hooks-Übung aus dem Unterricht (Folie 09/32; Custom Hooks allgemein 09/26-28):

```
export function useLocalStorage(key: string): ReturnValue {
  // Die Funktion in useState läuft nur beim ersten Render: localStorage wird nur einmal gelesen
  const [value, setStateValue] = useState(() => {
    const savedValue = localStorage.getItem(key)
    if (savedValue === null) {
```

```
      return ''
    }
    return savedValue
  })

  // Jede Änderung sofort speichern, damit der Wert auch nach einem Reload noch da ist
  function setValue(newValue: string) {
    setStateValue(newValue)
    localStorage.setItem(key, newValue)
  }

  return { value, setValue }
}
```

- **Lesen:** einmal beim ersten Render. Weil `useState` eine Funktion bekommt (Lazy Initial Value, Folie 09/8), liest React den Speicher nur beim ersten Render, nicht bei jedem (`useLocalStorage.ts:14-20`).
- **Schreiben:** `setValue()` setzt den State und speichert sofort (`useLocalStorage.ts:23-26`). Jeder Tastendruck im Namensfeld ist damit gespeichert, auch nach der Rückkehr vom Tisch.
- Der Hook gibt ein Objekt `{ value, setValue }` zurück, wie `useToggle` in Folie 09/28. Der `UserProvider` legt beides in den Context.
- Das Präfix `scrum-poker:` verhindert Kollisionen mit anderen Apps auf derselben Adresse (z. B. anderen Projekten auf `localhost:5173`).

Die `participantId` speichern wir bewusst **nicht**. Nach einem Reload soll man nicht an den Tisch zurückkehren, sondern auf der Startseite landen (Folie 07/6). Eine frühere Fassung der App hatte dafür `sessionStorage`; mit dem geänderten Leistungsnachweis ist das weggefallen.

---

11. Responsive Design und Box-Modell

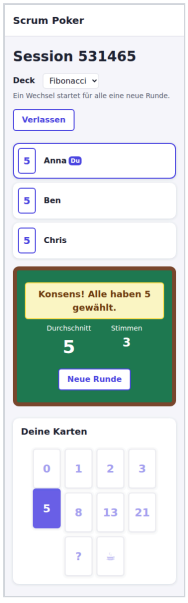
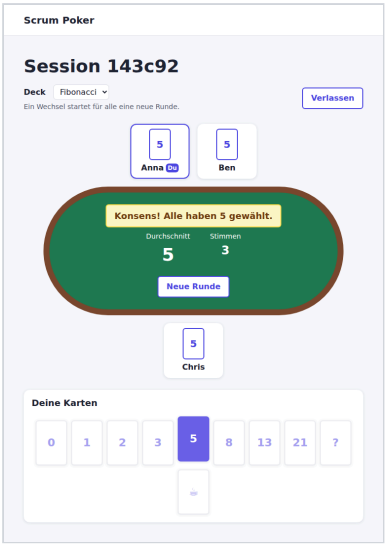
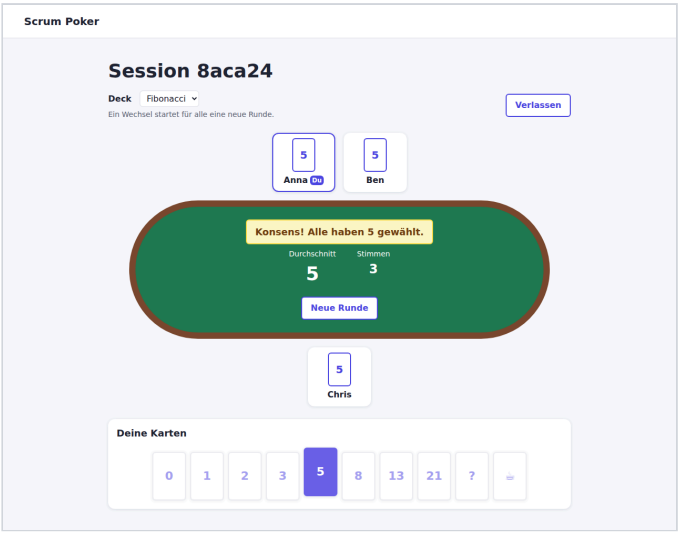
Eine CSS-Datei pro Komponente (Folie 08/19): `CardDeck.tsx` importiert `CardDeck.css`, `HomePage.tsx` importiert `HomePage.css` usw. Die Klassennamen haben den Namen der Komponente als Präfix (`card-deck__cards`, `playing-card--selected`), ähnlich wie in Folie 04/41 und 08/19, so kommen sich die Dateien nicht in die Quere. Für die ganze App gilt nur `client/src/index.css` aus der Vorlage, ergänzt um Überschriften und die Buttons, die überall vorkommen. Die Farben sind feste Werte; welche Farbe wofür steht, sagt der Kommentar oben in `index.css`.

Mobile First. Die Grundregeln gelten für schmale Bildschirme, die Media Query `@media (min-width: 768px)` (Folie 05/7) ergänzt die Änderungen ab Tablet-Breite. Es gibt einen Breakpoint: **768 px**. Die Folien zeigen das Beispiel mit `max-width`; wir haben `min-width` gewählt, damit die Regeln fürs Handy die einfachen sind.

Bereich	Mobil (< 768 px)	Ab 768 px	Technik
Startseite	Karten untereinander	zwei Spalten 2 : 3	**CSS Grid** (Folie 04/37-44): `grid-template-columns: minmax(0, 2fr) minmax(0, 3fr)` (`HomePage.css:49-54`)
Pokertisch	alle Plätze als Liste, Tisch darunter (`order: 1`, Folie 05/11)	Plätze oben und unten, ovaler Tisch dazwischen	**Flexbox** (Folie 04/24-35) `flex-direction: column` → `row` + `flex-wrap` (`PokerTable.css:5-35`, `PokerTable.css:41-62`)
Teilnehmerplatz	Karte links, Name rechts	Karte über dem Namen	Flexbox (`ParticipantSeat.css:4-14`, `ParticipantSeat.css:67-83`)

Bereich	Mobil (< 768 px)	Ab 768 px	Technik
Handkarten	brechen in mehrere Zeilen um	grössere Karten	Flexbox mit `flex-wrap: wrap` (`CardDeck.css:16-22`, `CardDeck.css:61-66`)
Inhalt	Rand `24px 16px`	Rand `40px`, max. 960 px breit, zentriert	`App.css:15-29`

- Grid für das Seitenlayout mit Spalten, Flexbox für Reihen von Elementen (Buttons, Karten, Plätze), die umbrechen dürfen.
- `minmax(0, …)` verhindert, dass ein langer Inhalt (z. B. ein langer Name) eine Grid-Spalte breiter macht als vorgesehen.
- `<meta name="viewport" content="width=device-width, initial-scale=1.0">` (`client/index.html:6`, Grundgerüst aus Folie 04/6) sorgt dafür, dass Handys die echte Breite verwenden und die Media Queries greifen.
- Angesehen haben wir die Breiten mit der Mobile View der DevTools (Folie 05/5).

Mobil (360 px)	Tablet (768 px)	Desktop (1280 px)
		

Box-Modell (Folie 04/19). Jedes Element besteht aus Inhalt, `padding`, `border` und `margin`. Global gilt `box-sizing: border-box` (`index.css:12-17`, so steht es in der Vorlage; die kurze Form `* { box-sizing: border-box; }` ist der Mini-CSS-Reset aus Folie 05/8). Damit enthalten `width` und `height` Padding und Rahmen. Ein Platz mit `width: 120px` bleibt also 120 px breit, egal wie dick sein Rahmen ist. Mit dem Standard `content-box` würden Padding und Rahmen dazugerechnet, und Layouts würden überlaufen. Die Einheiten sind `px`, `%` und `vh` aus Folie 04/17.

Reihenfolge der CSS-Dateien. `main.tsx` lädt `index.css` zuerst, danach kommen über die Komponenten deren CSS-Dateien (`main.tsx:4-6`). Bei gleich starken Regeln gewinnt die spätere. So kann `SessionListItem.css` den allgemeinen Button für «Löschen» rot machen (`.button--danger`). In der Vorlage steht der Import von `index.css` hinter dem von `App`; wir haben ihn genau aus diesem Grund nach vorne gezogen.

Verdeckte Karten. Ein Platz zeigt eine von drei Karten: leer (gestrichelt), verdeckt (einfarbige Rückseite) oder aufgedeckt (Wert). Das sind drei CSS-Klassen (`ParticipantSeat.css:21-47`), die

`ParticipantSeat` je nach Zustand setzt. Der Wert der anderen steht vor dem Aufdecken nicht einmal im HTML, weil der Server ihn gar nicht schickt.

Gewählte Karte. Die gewählte Handkarte rutscht mit `position: relative` und `top: -8px` etwas nach oben (Positionierung aus Folie 04/22, `CardDeck.css:43-49`).

Bedienung mit Tastatur und Screenreader (Auswahl): echte `<button>`- und `<select>`-Elemente, jedes Eingabefeld mit `<label htmlFor>`, ein sichtbarer Fokusrahmen mit `:focus` (Folie 04/14, `index.css:54-60`), `role="alert"` für Fehler und `role="status"` für Ladeanzeigen. Mehr ARIA haben wir bewusst weggelassen, weil es kein Kursstoff ist.

---

12. Wie wir geprüft haben

Die Folien behandeln keine automatischen Tests, und Test-Libraries wären zusätzliche Libraries (Folie 09/37). Im Projekt gibt es darum keine Testdateien. Geprüft haben wir so:

Prüfung	Befehl / Vorgehen	Was sie findet
Typprüfung und Build	`pnpm build` in `client/` (`tsc -b`, dann `vite build`) und in `server/` (`tsc -b`)	Typfehler. TypeScript 6 prüft standardmässig streng (`strict`), `any` taucht im Code nicht auf.
Lint	`pnpm lint` in beiden Ordnern (oxlint wie in der Vorlage, Folie 09/37)	z. B. fehlende Dependencies im `useEffect`, unbenutzte Variablen
Formatierung	`pnpm format:check` in beiden Ordnern (Prettier, Folie 09/37)	uneinheitlich formatierter Code
Konsole	Browser-Konsole und Terminal während des Durchklickens offen (Folie 09/37)	Warnungen von React, z. B. fehlender `key`
Durchklicken	zwei Browserfenster (eines davon privat) mit verschiedenen Namen, Ablauf aus Kapitel 1	alle Funktionen, inklusive verdecktes Abstimmen, automatisches Aufdecken, Deck-Wechsel, Löschen, Verlassen
Fehlerfälle	Server mit `Ctrl+C` stoppen, Session löschen während jemand am Tisch sitzt, leerer Name, Tab schliessen und 20 s warten	Lade- und Fehlerzustände aus Kapitel 7 und 8
Breiten	DevTools Mobile View mit 360, 768 und 1280 px (Folie 05/5)	horizontales Scrollen, überlappende Elemente
StrictMode	im Dev-Modus (`pnpm dev`) beitreten und im zweiten Fenster die Plätze zählen	genau ein Teilnehmer pro Fenster
Ungültige Requests	z. B. `curl -X PUT -H 'Content-Type: application/json' -d '{"value":"99"}' http://localhost:3000/api/sessions/<id>/participants/<pid>/vote`	Server antwortet 400 statt die Karte zu speichern

Die Screenshots in dieser Dokumentation stammen aus diesen Durchläufen (gebauter Client, drei Breiten). Zusätzlich hat die KI ausserhalb des abgegebenen Projekts einen Browser-Test (Playwright) laufen lassen, der die App von aussen bedient; er gehört nicht zur Abgabe.

---

13. Lokale Inbetriebnahme

Voraussetzungen

Werkzeug	Version	Prüfen mit
Node.js	**24** (LTS, wie in Folie 03/17)	`node -v`
pnpm	**12**	`pnpm -v`
Browser	aktueller Chrome, Firefox, Edge oder Safari	–

Beide Versionen stehen auch in `client/package.json` und `server/package.json` (`engines` und `packageManager`). Node 24 braucht es auch, weil der Server die `.ts`-Dateien direkt ausführt (Kapitel 2).

Schritt für Schritt

1. **Node.js 24 installieren** von https://nodejs.org (LTS) oder mit einem Versionsmanager wie `fnm` (Folie 03/5). Danach `node -v` prüfen.
2. **pnpm 12 installieren**, z. B. mit `npm install -g pnpm@12`, und `pnpm -v` prüfen.
3. **Projekt holen**: entweder das Code-ZIP entpacken oder das Repository klonen, dann in den Projektordner wechseln:

   ```
   cd scrum-poker            # aus dem ZIP
   # oder: cd abgaben/web-technology/abgabe/code
   ```

4. **Server starten** (erstes Terminal):

   ```
   cd server
   pnpm install
   pnpm dev
   ```

   Im Terminal erscheint `Scrum Poker API läuft auf http://localhost:3000`. Dank `node --watch` startet der Server bei jeder Code-Änderung neu.
5. **Client starten** (zweites Terminal, wieder im Projektordner):

   ```
   cd client
   pnpm install
   pnpm dev
   ```

   Vite meldet die Adresse http://localhost:5173.
6. **Im Browser öffnen:** http://localhost:5173. Zum Ausprobieren ein zweites Fenster (oder ein privates Fenster mit anderem Namen) öffnen und derselben Session über die Liste beitreten.

Dienst	Adresse	Port
Client (Vite-Dev-Server)	http://localhost:5173	5173
API (Express)	http://localhost:3000/api/sessions	3000
API über den Proxy	http://localhost:5173/api/sessions	5173

Weitere Befehle, jeweils im Ordner `client/` bzw. `server/`:

Befehl	`client/`	`server/`
`pnpm dev`	Vite mit Hot Reload	Server mit `node --watch`

Befehl	`client/`	`server/`
`pnpm build`	Typprüfung und Build nach `client/dist`	nur Typprüfung (`tsc -b`)
`pnpm lint`	oxlint	oxlint
`pnpm format` / `pnpm format:check`	Prettier: formatieren / nur prüfen	dasselbe
`pnpm preview`	Vorschau des Builds	–
`pnpm start`	–	Server ohne Watch

Produktivbetrieb ohne Vite: zuerst `pnpm build` in `client/`, dann in `server/` `NODE_ENV=production pnpm start` (Windows PowerShell: `$env:NODE_ENV='production'; pnpm start`). Der Server liefert dann auch den Client aus, alles läuft unter http://localhost:3000. Beenden jeweils mit `Ctrl+C` im Terminal.

Troubleshooting

Problem	Ursache und Lösung
Rote Meldung «Server nicht erreichbar.»	Der Express-Server läuft nicht (erstes Terminal) oder ist abgestürzt. Im Terminal des Servers nach Fehlern schauen und `pnpm dev` neu starten.
`EADDRINUSE: address already in use :::3000`	Ein alter Server läuft noch. Prozess beenden: `lsof -i :3000` (macOS/Linux) bzw. `netstat -ano | findstr :3000` (Windows). Ein anderer Port geht mit `PORT=3002 pnpm dev`, dann muss aber auch der Proxy in `client/vite.config.ts` auf 3002 zeigen.
Vite startet auf 5174 statt 5173	Port 5173 ist belegt, Vite weicht dann selbst aus. Die App funktioniert trotzdem, einfach die gemeldete Adresse öffnen.
`ERR_UNKNOWN_FILE_EXTENSION ".ts"` oder Syntaxfehler beim Start des Servers	Node-Version zu alt. `node -v` muss 24 sein.
`pnpm: command not found`	Schritt 2 ausführen oder das Terminal neu öffnen.
`ERR_PNPM_…` bei `pnpm install`	Mit `npm` oder `yarn` installiert? `node_modules` im betroffenen Ordner löschen und nur `pnpm install` verwenden.
Alle Sessions weg	Der Server hat neu gestartet (z. B. nach einer Code-Änderung durch `node --watch`). Die Daten liegen nur im Arbeitsspeicher, das ist so gewollt.
Eine Person bleibt nach dem Schliessen des Tabs oder einem Reload am Tisch	Sie verschwindet nach höchstens 20 s (Heartbeat-Timeout, Kapitel 8).

---

14. Abgabe

Zwei gleichwertige Wege:

1. **ZIP-Datei:** Im Projektordner

   ```
   ./scripts/make-submission.sh
   ```

   ausführen. Es erzeugt `scrum-poker-code.zip` mit `git archive` aus dem committeten Stand: den Quellcode von `client/` und `server/` ohne `node_modules` und `dist`, dazu das README

und den Leitfaden fürs Fachgespräch (`docs/FACHGESPRAECH.md` / `.pdf` / `.html`). Diese Dokumentation (PDF und HTML) wird daneben abgegeben. Nach dem Entpacken laufen in beiden Ordnern `pnpm install`, `pnpm lint` und `pnpm build` ohne weitere Schritte. Das ZIP selbst ist nicht im Git (`.gitignore`).

2. **Git-Link** auf den öffentlichen Abgabe-Ordner (Dokumentation, Leitfaden, ZIP und Quellcode): https://github.com/gotazlib/abgaben/tree/main/web-technology/abgabe. Den Link bekommt der Dozent bis Ende November (Folie 01/5).

PDF und HTML dieser Dokumentation und des Leitfadens werden mit `./scripts/build-pdf.sh` aus den Markdown-Dateien erzeugt (nach jeder Änderung erneut ausführen).

Bearbeitbare Version: `DOKUMENTATION.docx`. Nach dem Bearbeiten in Word Inhaltsverzeichnis aktualisieren und als PDF speichern. Die Word-Datei nutzt die HWZ-Vorlage und wird mit `./scripts/build-docx.sh` aus dieser Markdown-Datei erzeugt. Das überschreibt Änderungen, die in Word gemacht wurden.

---

15. KI-Einsatz

Offenlegung

Die App wurde zum grössten Teil mit KI generiert. Code, CSS, Screenshots und diese Dokumentation hat ein KI-Agent geschrieben. Die Gruppe hat die Aufgaben formuliert, Anforderungen aus der Aufgabenstellung und den Kursfolien vorgegeben, Ergebnisse geprüft und die Änderungen freigegeben.

Werkzeug	Modell	Einsatz
Claude Code (CLI von Anthropic), mehrere Agenten mit je einem Pull Request pro Arbeitspaket	Claude Opus	gesamte Umsetzung, siehe unten

Arbeitspaket	Was die KI gemacht hat
Grundgerüst	erstes Projekt-Setup mit Vite und Express, TypeScript-Konfiguration, gemeinsame Types
Server	REST-API, Store, Validierung, Heartbeat-Strategie
Startseite	Komponenten, Context, Polling, Löschen, Screenshots
Pokertisch	Beitreten/Verlassen mit StrictMode, Karten, Decks, Aufdecken, Durchschnitt, Screenshots
CSS	Mobile-First-Layout, Tisch, Karten
Abnahme	Prüfung gegen die Aufgabenstellung, automatische Tests, Screenshots für Desktop/Tablet/Mobile
Dokumentation	diese Dokumentation, Leitfaden fürs Fachgespräch `docs/FACHGESPRAECH.md`, PDF-, HTML-, DOCX- und Abgabe-Scripts
Vereinfachung (auf Wunsch der Gruppe, Oktober 2026)	Die Gruppe hat zurückgemeldet, dass der erste Code für Anfänger zu schwer zu erklären war. Auf ihren Wunsch hat die KI einen Vereinfachungsplan geschrieben (keine cleveren Abstraktionen) und den Code danach umgebaut. Mit der späteren Angleichung an die Folien kamen der Custom Hook `useLocalStorage` (Folie 09/32) und Generics bei `useState` (Folie 07/16) wieder dazu, weil die Folien sie zeigen. Ein KI-Reviewer hat jeden Teil aus Sicht eines Anfängers geprüft. Danach wurden Dokumentation, Leitfaden und Screenshots angepasst.

Arbeitspaket	Was die KI gemacht hat
Angleichung an Kursfolien und Projektvorlage (Oktober 2026)	Nach den Foliensätzen 7 und 9 (geänderter Leistungsnachweis, Tipps zum Projekt) und der Projektvorlage «scrum-poker-hwz» hat die KI den Code Datei für Datei mit den Folien abgeglichen und umgebaut: Ansichtswechsel mit State statt React Router, `useLocalStorage`, keine Klasse (Error Boundary) und keine zusätzlichen Libraries mehr (Test-Libraries, `tsx`, `concurrently`), Client wie die Vorlage (oxlint, Prettier, Ordner, Node 24/pnpm 12), Server als eigenes Projekt, `type` statt `interface`, eine CSS-Datei pro Komponente. Die automatischen Tests sind dabei aus dem Projekt verschwunden (Kapitel 12). Danach wurden Dokumentation, Leitfaden, Screenshots und alle `Datei:Zeile`-Verweise neu erstellt bzw. geprüft.

Alle Arbeitsschritte sind in der Git-Historie nachvollziehbar.

Von Hand geschrieben oder angepasst

TODO Gruppe: Hier ehrlich auflisten, was ihr selbst geschrieben, geändert oder entschieden habt (Datei, Art der Änderung). Wenn nichts von Hand geschrieben wurde, das so stehen lassen und stattdessen beschreiben, was ihr geprüft und entschieden habt (z. B. den Wunsch nach einfacherem Code und nach Code auf dem Niveau der Folien). Nichts erfinden.

Wie die Gruppe sichergestellt hat, dass sie den Code versteht

TODO Gruppe: Beschreiben, wie ihr den Code durchgearbeitet habt (z. B. Leitfaden docs/FACHGESPRAECH.md mit der Aufteilung im Kapitel «Aufteilung für die Gruppe» durchgearbeitet, Übungsaufgaben gelöst, Code gemeinsam besprochen).

Gruppenmitglieder und Aufteilung

TODO Gruppe: Namen aller Gruppenmitglieder und wer welchen Teil des Codes zuerst vorbereitet hat. Im Fachgespräch muss trotzdem jede Person alles erklären können.